
Describe the Task First, Ask Last? Query Placement and Prefix-Cache Reuse in Small Long-Context Models

Qinzhen Ma
Rice University
qm18@rice.edu

Jialin Wu
UC San Diego

Shichen Tang
Rice University

Abstract

Exact prefix caching reuses the key-value cache of a shared document only when each question comes after it, yet practitioner guides disagree on whether questions should precede long documents. We measure what this cache-friendly, query-last layout costs. On three open instruction-tuned models with 3–4B parameters, we compare six prompt layouts on multi-key needle retrieval with distractors at 3.5k–16k tokens and five of them on three LongBench tasks, and we time prefill when eight questions share one document. Query-first prompts are the clear failure: they cost the two models that are not at ceiling 18.3 and 25.0 points of needle accuracy, always by returning a distractor’s value, and lose 10.7 LongBench points relative to query-last. Repeating the question on both sides of the document gains at most 4.2 needle points for any model, which is not significant after correction, scores 3.2 points lower on LongBench, and needs $6.9\times$ the prefill time for eight questions about a 16k-token document. A query-agnostic task description placed before the document, which keeps the document cacheable, barely changes needle accuracy and raises LongBench scores by 4.3 points over query-last, while a length-matched irrelevant preamble does not. In our setting, describing the task before the document and asking the question after it gives the best LongBench average of the five layouts we test, at nearly the prefill cost of query-last.

1 Introduction

Many applications ask a series of questions about one long document [Li et al., 2025]. Serving systems make this workload cheap by reusing precomputed key-value (KV) states across requests [Gim et al., 2024, Kwon et al., 2023, Zheng et al., 2024]. Because each cached state depends on every earlier token, exact prefix reuse keeps the document’s cache only when the question follows the document, as in the query-last layout (QL); query-first (QF) and query-both (QB) layouts recompute the document for every question.

Practitioner advice on where the question belongs conflicts: OpenAI recommends instructions both above and below long context [MacCallum and Lee, 2025], whereas Anthropic recommends documents first and queries last [Anthropic, 2026]. Controlled evidence is mixed. Query-aware contextualization, which places the query before and after the data, makes key-value retrieval nearly perfect but barely helps multi-document QA [Liu et al., 2024]. Gupta et al. [2024] find appended instructions considerably worse than prepended or sandwiched ones on GPT-4o and GPT-4-Turbo, whereas Park et al. [2025] find no consistent winner between question-first and standard order on BABILong. Repeating the prompt improves non-reasoning models in most tested comparisons, with the cost framed as extra prefill only [Leviathan et al., 2025]. None of these studies accounts for lost cache reuse when many questions share one document.

This paper is a measurement study, not a new method. We ask whether cache-friendly layouts lose accuracy relative to query-aware ones, and whether a query-agnostic task preamble before

Table 1: Prompt layouts. The bar separates the prefix shared by all questions about a document from the per-question suffix. Segments are separated by blank lines, except that F follows Q after a single newline. The layout string is the user turn, preceded by the fixed system message. TP-POST shares the prefix of QL.

Layout	Segments	Cacheable
QL	$D \mid Q F$	✓
QF	$Q D F$	
QB	$Q D Q F$	
TP	$P D \mid Q F$	✓
IRR	$P_{\text{irr}} D \mid Q F$	✓
TP-POST	$D \mid P Q F$	✓

the document (TP) recovers any such loss while keeping the prefix cacheable. This layout is not new: LongBench templates and RULER’s NIAH template already use it [Bai et al., 2024, Hsieh et al., 2024]. It is also the zero-compression analogue of task-guided KV compression, where a task description stands in for the unknown query when selecting cache entries [Corallo et al., 2025, Park and Hwang, 2026]. To our knowledge, its effect with a full, uncompressed cache has not been isolated against query-aware layouts. Prior evidence suggests only a partial effect: reminders that do not restate the question help less than ones that do [Agrawal et al., 2024]. Two controls separate the effects of extra tokens, task content, and position: a length-matched irrelevant preamble (IRR) and the same task preamble placed after the document (TP-POST).

Contributions.

- A paired comparison of four cacheable and two query-aware layouts on three open instruction-tuned models of at most 4B parameters, using multi-key needle retrieval with distractors and three LongBench tasks (without TP-POST). Query-first hurts most: it loses 14.4 needle points and 10.7 LongBench points against QL, while QB averages within a few points of QL on both.
- A decomposition of the preamble effect using the IRR and TP-POST controls and a NIAH error taxonomy. A task preamble before the document adds 4.3 LongBench points over QL and 5.3 over the length-matched IRR control, but barely changes standard needle accuracy, where QL is already close to QB.
- A measurement of the prefill cost of query-aware layouts when several questions share one document. QB needs $5.1\text{--}6.9\times$ the total prefill time of cached QL or TP for eight questions about 4k–16k-token documents.

Related work. Position-independent caching [Yao et al., 2025, Hu et al., 2025] reuses KV states of non-prefix segments and could make query-first layouts partially cacheable; we do not evaluate it. Query-aware KV compression such as SnapKV [Li et al., 2024] degrades when one context serves many queries [Li et al., 2025, Kim et al., 2025]; we keep the cache uncompressed. The effect of question-first order depends on the model [Ling et al., 2025], so we report models separately.

2 Experimental Setup

2.1 Layouts

Prompts combine a document D , a question Q , a query-agnostic format line F , and optionally a preamble (Table 1; strings in Appendix A). In TP, the fixed task description P names no key, entity, or question. IRR replaces P with token-count-matched neutral prose, and TP-POST moves P after D , where causal attention keeps it from shaping the document’s encoding.

2.2 Tasks

Multi-key NIAH. Following RULER [Hsieh et al., 2024], a haystack of Paul Graham essays [Kamradt, 2023] hides ten needles (“The special magic number for *key* is: *number*.”) at sentence boundaries at relative depths $(i + 0.5)/10$. Keys (adjective–noun pairs) share no words and the 7-digit numbers

Table 2: Accuracy (%) by layout. NIAH: multi-key needle retrieval pooled over 3.5k/8k/16k-token documents (120 queries per cell). LongBench: macro-average over three tasks at an 8k-token budget. QL, TP, TP-POST, and IRR keep the document prefix cacheable; QF and QB do not.

Model	NIAH (10 keys, 9 distractors)						LongBench (3 tasks)				
	QL	TP	TP-POST	IRR	QF	QB	QL	TP	IRR	QF	QB
Qwen2.5-3B	95.8	96.7	98.3	93.3	77.5	100.0	48.5	55.5	43.3	36.9	46.8
Phi-4-mini	95.8	96.7	92.5	96.7	70.8	97.5	57.3	59.4	58.6	47.0	53.6
Qwen3-4B-2507	100.0	100.0	100.0	100.0	100.0	100.0	61.8	65.7	62.6	51.6	57.5

are distinct, so each query faces nine same-format distractors. We query needles $i \in \{0, 2, 5, 7, 9\}$ in 8 haystacks at each of 3.5k, 8k, and 16k document tokens (120 queries per model and layout; Appendix B).

LongBench. For passage_retrieval_en, hotpotqa, and multifieldqa_en [Bai et al., 2024], we truncate only the context to an 8k-token total budget: QA contexts lose middle tokens, and passage-retrieval contexts lose whole middle paragraphs, skipping items that would lose the gold paragraph. We use the first 40 surviving items per task in dataset order and all layouts except TP-POST (Appendix C).

2.3 Models and inference

We run Qwen2.5-3B-Instruct [Yang et al., 2024], Qwen3-4B-Instruct-2507 [Yang et al., 2025], and Phi-4-mini-instruct [Abouelenin et al., 2025] in Hugging Face Transformers (bf16, batch size 1, greedy decoding). Prompts use each chat template with a fixed system message (Appendix A.2). Emulating exact prefix caching [Kwon et al., 2023, Zheng et al., 2024], each request reuses the KV cache of its longest common token prefix with the previous request on the same document, so accuracy reflects the cached path. Re-decoding QL and QB on each length’s first haystack without reuse (30 pairs per model) gives identical outputs for 89 of 90 pairs; in the one mismatch, a Qwen2.5-3B QL query, the cached path returns a distractor and recomputation the correct number (Appendix D). The other layouts were not re-decoded.

2.4 Metrics and statistics

NIAH outputs are labeled by their first 7-digit number as correct, distractor (another needle’s number), hallucination (any other number), or format failure (none). LongBench outputs are scored on their first line with qa-F1 or the retrieval score (Appendix C). Comparisons are paired over identical documents and questions. The pre-declared primary family has six tests (NIAH accuracy TP–QL and QB–QL per model, pooled over lengths), with haystack-cluster bootstrap 95% CIs and two-sided sign-flip tests, Holm-corrected (Appendix E). All other contrasts, including LongBench (item-level paired bootstrap), are exploratory.

2.5 Latency

We ask Qwen3-4B-Instruct-2507 $M=8$ NIAH questions about one 4k-, 8k-, or 16k-token document and time GPU-synchronized prefill over each question’s uncached tokens. QL and TP reuse the document prefix after the first question; QB recomputes every prompt. We report medians over three repetitions with alternating layout order (Appendix B).

3 Results

Needle retrieval. The cache-friendly layout loses little on multi-key retrieval (Table 2, Figure 1). Pooled over models, QB beats QL by 1.9 points (95% CI $[0.0, 3.9]$), and the largest per-model gap is 4.2 points for Qwen2.5-3B ($[1.7, 7.5]$, Holm-adjusted $p = 0.39$). No primary test survives correction, but this is not evidence of equivalence: only 5 of that model’s 24 haystacks differ, all favoring QB, and Qwen3-4B, at 100% in every layout, cannot show an effect. Query-first fails in exploratory contrasts: QF loses 18.3 points for Qwen2.5-3B and 25.0 for Phi-4-mini (14.4 pooled, $[-17.8, -11.1]$, $p < 0.001$), and 25.0 pooled at 16k tokens. Every QF error returns another needle’s number (Table 6), never a hallucination or format failure. The task preamble changes little (+0.6

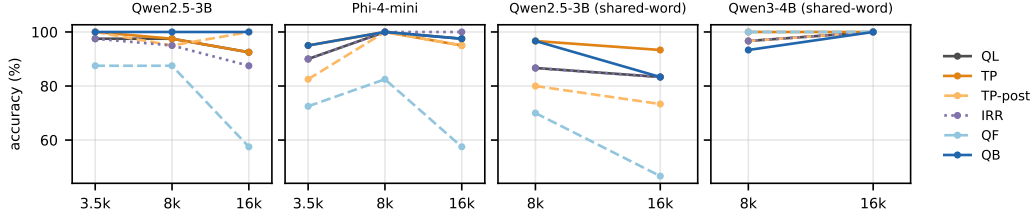


Figure 1: Multi-key NIAH accuracy by document length and layout. Left: standard task (40 queries per point); Qwen3-4B, at 100% in every cell, is not shown. Right: exploratory variant with shared-word keys (30 queries per point). QF is the only layout that degrades sharply (up to 42.5 points below QB on the standard task), and QL stays within 7.5 points of QB in every standard-task cell.

points pooled, $[-1.1, 2.2]$; for Qwen2.5-3B it recovers 20% of the 4.2-point QB gap), and neither IRR nor TP-POST moves any model by more than 3.3 points.

LongBench. On real tasks the ordering is sharper (Table 2, right). Macro-averaged over models and tasks, TP scores 60.2, QL 55.9, IRR 54.8, QB 52.6, and QF 45.1. The task preamble improves on QL by 4.3 points (95% CI $[2.0, 6.6]$, 360 paired items) and on the length-matched IRR control by 5.3 points ($[2.6, 8.0]$), so extra tokens alone do not explain the gain. TP is never below QL in any of the nine model–task cells (Table 7). QF again loses 10.7 points ($[-14.6, -6.9]$), and QB does not help on average ($-3.2, [-6.7, 0.2]$): repeating the question helps multi-hop hotpotqa (+6.1) but hurts multifieldqa_en (-5.0) and passage retrieval (-10.8). LongBench contrasts are exploratory and uncorrected.

Harder distractors. To probe the ceiling, an exploratory variant builds the ten keys from two adjectives and five nouns, so each queried key shares a word with five distractors (8k and 16k tokens, 60 queries per layout; Table 9). Qwen3-4B still scores 96.7–100% in every layout. For Qwen2.5-3B, QF falls to 58.3% against 85.0% for QL, while TP reaches 95.0% (+10.0 points, $[1.7, 16.7]$, $p = 0.07$), above QB at 90.0%. The same preamble placed after the document (TP-POST) scores 76.7%, so under key interference the preamble seems to help only before the document, though moving it also changes its coherence. With 60 queries on one model, this is suggestive rather than conclusive.

The price of query awareness. Under exact prefix reuse, every question after the first pays only for its own suffix. For a 16k-token document, a later question costs 0.075 s of prefill with QL or TP, against 5.2 s with QB, a $70\times$ difference (Table 8). Over eight questions, total prefill time drops from 41.6 s to 6.1 s ($6.9\times$), and from 15.3 s to 2.4 s at 8k ($6.3\times$). The task preamble costs little: its 87 extra tokens raise total prefill time by 3% over QL at 16k. The ratio grows with document length and approaches the per-question ratio as more questions share a document. These batch-1 timings on one consumer GPU indicate only the rough size of the saving; absolute times and exact ratios depend on kernels and batching.

Limitations. Our study is deliberately small. We test three open instruction-tuned models of at most 4B parameters, documents of at most 16k tokens, and 40 LongBench items per task, so effects that appear only at larger scales or longer contexts are out of scope. All accuracy numbers use greedy decoding with one fixed system message and one wording per layout, and other wordings could change them. Qwen3-4B is at or near ceiling on both needle variants and cannot show layout effects there, and with 24 haystacks per model the primary tests have little power for differences of a few points. TP-POST reuses P verbatim, including “The document below”, so this control changes the preamble’s coherence as well as its position. The NIAH task preamble reveals the needle format, which is information a deployed system would know when it builds the cache for a known task type, but it is more specific than a generic instruction. Latency comes from a research implementation, so absolute times do not transfer to optimized serving engines.

References

- Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, et al. Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *arXiv preprint arXiv:2503.01743*, 2025.
- Devanshu Agrawal, Shang Gao, and Martin Gajek. Can’t remember details in long documents? You need some R&R. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12692–12704, 2024. doi: 10.18653/v1/2024.findings-emnlp.742.
- Anthropic. Prompting best practices: Long context prompting. Claude Platform Documentation, 2026. URL <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>. Accessed 2026-09-14.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, 2024. doi: 10.18653/v1/2024.acl-long.172.
- Giulio Corallo, Orion Weller, Fabio Petroni, and Paolo Papotti. Beyond RAG: Task-aware KV cache compression for comprehensive knowledge reasoning. *arXiv preprint arXiv:2503.04973*, 2025.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Lavanya Gupta, Saket Sharma, and Yiyun Zhao. Systematic evaluation of long-context LLMs on financial concepts. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1163–1175, 2024. doi: 10.18653/v1/2024.emnlp-industry.88.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.
- Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 24391–24402, 2025.
- Greg Kamradt. Needle in a haystack. GitHub repository, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack.
- Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Prompt repetition improves non-reasoning LLMs. *arXiv preprint arXiv:2512.14982*, 2025.
- Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. SCBench: A KV cache-centric analysis of long-context methods. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

- Zhan Ling, Kang Liu, Kai Yan, Yifan Yang, Weijian Lin, Ting-Han Fan, Lingfeng Shen, Zhengyin Du, and Jiecao Chen. LongReason: A synthetic long-context reasoning benchmark via context expansion. *arXiv preprint arXiv:2501.15089*, 2025.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.
- Noah MacCallum and Julian Lee. GPT-4.1 prompting guide. OpenAI Cookbook, 2025. URL https://developers.openai.com/cookbook/examples/gpt4-1_prompting_guide. Published 2025-04-14.
- Joon Park, Kyohei Atarashi, Koh Takeuchi, and Hisashi Kashima. Emulating retrieval augmented generation via prompt engineering for enhanced long context comprehension in LLMs. *arXiv preprint arXiv:2502.12462*, 2025.
- Wonpyo Park and Seung-won Hwang. TaskPress: Query-agnostic KV cache compression via task-guided pruning. *arXiv preprint arXiv:2608.03276*, 2026. To appear at EMNLP 2026.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696098.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

A Prompt Strings

All strings are copied from the experiment script. Python f-string notation is used: `\n` is a newline and a name in braces is a substituted field. The preambles, format lines, irrelevant-text pool, and key vocabularies are each a single line with single spaces. They are wrapped here for typesetting, and every line break inside those blocks stands for exactly one space.

A.1 Layouts

The layouts differ only in the user-turn content; D , Q , F , and P are identical across layouts for a given item.

```
D      = f"<document>\n{doc}\n</document>"
QL     = f"{D}\n\n{Q}\n{F}"
QF     = f"{Q}\n\n{D}\n\n{F}"
QB     = f"{Q}\n\n{D}\n\n{Q}\n{F}"
TP     = f"{P}\n\n{D}\n\n{Q}\n{F}"
IRR    = f"{P_irr}\n\n{D}\n\n{Q}\n{F}"
TPpost = f"{D}\n\n{P}\n\n{Q}\n{F}"
```

A.2 Chat template and decoding

Every prompt is a two-message conversation. The system message is the same for all models, tasks, and layouts:

```
You are a helpful assistant.
```

The user message is the layout string. We pin the system message because, without one, the Qwen2.5 template inserts its own default system prompt. The conversation is rendered by `apply_chat_template` with `add_generation_prompt=True`, `tokenize=False`, and `enable_thinking=False`. Rendering with and without the thinking flag gives identical text for all three models. The rendered string is tokenized with `add_special_tokens=False`, so no second BOS token is added. The rendered prompts are shown below with real line breaks, where `{USER}` is the layout string. The Qwen prompt ends with a newline after assistant, and the Phi-4-mini prompt is a single line.

Qwen2.5-3B-Instruct and Qwen3-4B-Instruct-2507:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
{USER}<|im_end|>
<|im_start|>assistant
```

Phi-4-mini-instruct:

```
<|system|>You are a helpful assistant.<|end|><|user|>{USER}<|end|><|assistant|>
```

Decoding is a greedy argmax loop that computes logits only at the last position. It stops at the token limit (16 for NIAH, 10 for `passage_retrieval_en`, 32 for the QA tasks) or at any end-of-sequence id from the tokenizer or the generation config (for Qwen3-4B, `<|endoftext|>` and `<|im_end|>`). Outputs are decoded with special tokens removed. The rendered chat header precedes the layout string, so it belongs to every reusable prefix.

A.3 Multi-key NIAH

```
needle = f"The special magic number for {key} is: {value}."
key     = f"{adjective}-{noun}"
Q       = f"Question: What is the special magic number for {key}?"
F       = "Answer with only the 7-digit number."
```

P for NIAH:

Task description: The document below is a collection of essays. Several short statements of the form "The special magic number for <key> is: <number>." are hidden at different places in it; each gives a different key a different 7-digit number. After the document you will be asked for the number of one of these keys. While reading, note every such statement and keep track of which number belongs to which key.

Adjectives (30):

amber crimson silver quiet rapid hollow golden misty bitter gentle frozen
velvet rustic lunar scarlet humble brisk ancient cobalt dusty fierce ivory
jagged lucky marble noble olive polar rosy sunny

Nouns (30):

harbor lantern falcon meadow compass violin glacier orchard canyon beacon
willow anchor pebble thicket summit cottage prairie quarry saddle tunnel
kettle ledger mirror needle otter parrot ribbon spindle turnip walnut

A.4 LongBench

`{input}` is the LongBench input field of the item. The script also defines `2wikimqa` with the strings below; Section 2 lists the subsets we report.

`passage_retrieval_en` (retrieval score, at most 10 new tokens):

```
Q = f"Summary: {input}"
F:
```

Which paragraph does the summary describe? Answer only with "Paragraph" followed by its number.

P:

Task description: The document below consists of numbered paragraphs, each starting with "Paragraph <number>:". After the document you will be given a short summary of exactly one of these paragraphs and asked which paragraph it describes. While reading, note the main topic and distinctive details of each paragraph together with its number.

hotpotqa (qa-F1, at most 32 new tokens):

Q = f"Question: {input}"

F:

Answer with only the short answer (for example a name, a date, or yes/no), with no explanation.

P:

Task description: The document below consists of several Wikipedia paragraphs, many of them unrelated to each other. After the document you will be asked a question whose answer requires combining facts from two different paragraphs, for example connecting a person, place, organization, or work mentioned in one paragraph to details given in another. While reading, keep track of the entities in each paragraph and how they are related. The answer will be a short phrase, such as a name, a date, or yes/no.

multifieldqa_en (qa-F1, at most 32 new tokens):

Q = f"Question: {input}"

F:

Answer as concisely as possible, using only the document.

P:

Task description: The document below is a long text from one of several domains, such as a research paper, a legal or policy document, a technical report, or a news article. After the document you will be asked a factual question whose answer is stated somewhere in it. While reading, pay attention to definitions, methods, results, names, quantities, and dates, and remember roughly where each piece of information appears. The answer should be concise and based only on the document.

2wikimqa (qa-F1, at most 32 new tokens):

Q = f"Question: {input}"

F:

Answer with only the short answer (for example a name, a date, or yes/no), with no explanation.

P:

Task description: The document below consists of several Wikipedia paragraphs. After the document you will be asked a question that requires reasoning over two or more paragraphs, for example comparing two people, films, or places (such as which came first or which is older), or following a chain of relations (such as the director of a film and then that director's place of birth). While reading, keep track of names, dates, places, and family or work relations. The answer will be a short phrase, such as a name, a date, or yes/no.

A.5 Irrelevant preamble and length matching

One pool of neutral prose with no digits, names, or task words serves every task:

The garden behind the old house is quiet in the early morning. Narrow paths of pale gravel wind between low hedges, and a small stone fountain stands where two of the paths meet. In spring the beds along the wall fill with tulips and irises, and later in the year the roses take their place. Birds gather in the tall trees at the far end, and on warm afternoons the air smells of cut grass. A wooden bench sits in the shade of a wide oak, and a

gardener comes by twice a week to trim the edges and sweep the leaves from the steps. When it rains, water collects in shallow pools on the paths and the colors of the flowers look deeper than usual. In the evening the light turns gold and then grey, and the garden becomes still again.

Length-matching rule. The pool is repeated three times and split on whitespace. For each tokenizer and task, P_{irr} is made of the first k words joined by single spaces. We choose k so that the token count of P_{irr} is closest to that of the task’s P , scanning k upward and stopping once the count exceeds $|P| + 5$; ties go to the smaller k . P_{irr} may therefore end mid-sentence. The longest selected prefix has 101 words, fewer than the pool’s 142, so the repetition is never used. Table 3 shows that $|P_{\text{irr}}|$ is always within one token of $|P|$. Qwen2.5 and Qwen3 give identical counts.

Table 3: Token counts of the fixed prompt segments. $|P|$ and $|F|$ are identical for the Qwen and Phi-4-mini tokenizers.

Task	$ P $	$ F $	$ P_{\text{irr}} $	
			Qwen	Phi-4-mini
NIAH	87	9	87	87
passage_retrieval_en	61	18	61	61
hotpotqa	98	23	97	98
multifieldqa_en	96	13	96	95
2wikimqa	112	23	112	112

B Haystack Construction

Corpus. The haystack source is 49 Paul Graham essays in plain text (644 KB), concatenated in file-name order with blank lines between them.

Needles. Haystack h at document length L uses Python’s `random.Random` seeded with $10007h + L$. Seeding does not depend on the model, so all models see the same keys, values, and start offset. The span is cut by token count, so Qwen2.5 and Qwen3 see identical documents, while Phi-4-mini’s document differs slightly. Ten adjectives and ten nouns are drawn without replacement and paired into ten keys, so no two keys share a word. Ten distinct values are drawn from $[1000000, 9999999]$. The needle budget is the summed token count of each needle with a leading space.

Filler and insertion. A start offset is drawn uniformly over character positions. From it we take $7L$ characters, tokenize them, keep the first L minus the needle budget tokens, and decode, so the document has approximately L tokens. Let n be the number of sentence ends (end positions of matches of `[. ! ?] \s+`). Needle $i \in \{0, \dots, 9\}$ is inserted, followed by a space, at sentence end number $\text{round}((i + 0.5)/10 \cdot (n - 1))$. Insertion runs from the last needle to the first so earlier positions stay valid. We query needles $i \in \{0, 2, 5, 7, 9\}$ (depths 0.05, 0.25, 0.55, 0.75, 0.95); the other nine are in-context distractors. The document is identical in all six layouts.

Scoring. Commas and whitespace between digits are deleted, and the first match of `(?<!\d)\d{7}(?! \d)` is the prediction. The label is **CORRECT** if it equals the queried value, **DISTRACTOR** if it equals another needle’s value in the same haystack, **HALLUC** for any other 7-digit number, and **FORMAT** if there is no match.

Query order and prefix reuse. The cache is cleared at the start of every haystack. The cacheable layouts run in blocks: QL for the five queried needles, then TP-POST, TP, and IRR. QF and QB then alternate per question. Under longest-common-token-prefix reuse, the first QL query prefills the document, TP-POST reuses that prefix, and TP and IRR each prefill their own prefix once. Each QB query reuses the QD prefix that was just computed for QF with the same question. Reuse always leaves at least the last prompt token to be recomputed. The recomputation check re-decodes QL and QB for the five queries of haystack $h=0$ at each length with the cache disabled. These decodes are used only for the agreement rate.

Latency haystacks. The latency runs use separate haystacks with seeds $h = 900 + r$ for repetition r , and questions about the first eight keys. The two warm-up prefills use a 2k-token haystack ($h=999$).

Exploratory hard variant. A harder NIAH variant uses keys from a 2×5 grid (two adjectives $\times$ five nouns, shuffled), so all ten keys are built from seven words and five of the nine distractor keys share a word with the queried key. It is seeded with $20011h + L$ and otherwise identical. We run it for Qwen3-4B and Qwen2.5-3B, with six haystacks at 8k and 16k tokens, all six layouts, and no recomputation check, and treat it as exploratory.

C LongBench Truncation and Item Selection

Data. We use the English LongBench (v1) JSONL files and keep items in file order.

Budget. For each item and tokenizer, the context budget is

$$B = 8000 - (|P| + 2|Q| + |F| + 80)$$

tokens. This covers the preamble of TP/IRR and the repeated question of QB at the same time, plus 80 tokens for the chat template, document tags, and separators. We checked with each tokenizer that every rendered prompt of every selected item has at most 8,000 tokens in all five layouts. Contexts with at most B tokens are used unchanged. Truncation changes only the context, and the same truncated context appears in all layouts.

QA tasks (hotpotqa, multifieldqa_en). Truncation happens in the middle. We decode the first $\lfloor B/2 \rfloor$ and the last $\lfloor B/2 \rfloor$ context tokens and join them with $\square \dots \square$. The gold answer need not appear verbatim in the context, so gold survival cannot be checked and no item is filtered.

Passage retrieval (gold-survival filter). For passage_retrieval_en, the context is split before every label of the form Paragraph <number>:. While the kept paragraphs exceed B tokens, we remove the paragraph at the middle index of the remaining list. Original numbering is kept. If the gold paragraph was removed, the item is skipped. Item selection finishes before any decoding.

Selection. Each task uses the first 40 items that survive, selected separately per tokenizer. Table 4 summarizes the result. Every passage-retrieval context exceeds the budget, so 61 (Qwen) or 59 (Phi-4-mini) items are scanned to keep 40. Items run round-robin across tasks, with layouts in the order QF, QB, QL, TP, IRR and the cache cleared per item. If a block’s time cap is reached, the tasks therefore lose items evenly (to within one item). Analyses keep only items with all five layouts completed.

Table 4: LongBench item selection at the 8k budget. Scanned: items read in file order to obtain 40 eligible items. Skipped: gold paragraph lost in truncation. Truncated: kept items whose context exceeded the budget. Context: mean logged context tokens of kept items (the budget B for middle-truncated QA contexts). Qwen2.5 and Qwen3 tokenizers give identical values.

Task	Tokenizer	Scanned	Skipped	Truncated	Context
passage_retrieval_en	Qwen	61	21	40	7,183
	Phi-4-mini	59	19	40	7,277
hotpotqa	Qwen	40	0	36	7,411
	Phi-4-mini	40	0	36	7,380
multifieldqa_en	Qwen	40	0	19	6,318
	Phi-4-mini	40	0	19	6,247

Post-processing and metrics. The same steps apply to every layout. We strip the raw output, keep its first line, strip that line, and remove a leading Answer: (case-insensitive). qa-F1 is token-level F1 after LongBench answer normalization (lowercasing and removing punctuation, articles, and extra whitespace), maximized over gold answers. The retrieval score is the fraction of numbers in the prediction that equal the gold paragraph number (0 if there are none). As secondary measures we log format failure (empty answer for QA; no Paragraph <number> for retrieval), normalized gold containment in the raw output, and the number of output tokens.

D Attention Implementation

FlashAttention is not available on our platform, so we register a custom attention function in Transformers. It has two paths, selected by the number n of query tokens in a forward pass.

Short queries ($n \leq 256$). These are decode steps and short uncached suffixes after a reused prefix. The function computes exact grouped-query attention with batched matrix multiplication. Each of the H_{kv} key/value heads is attended by its H/H_{kv} query heads, so key/value heads are never repeated. Attention scores are cast to float32 before the causal mask (aligned to the end of the cache) and the softmax.

Long prefill ($n > 256$). PyTorch scaled-dot-product attention is used with key/value heads repeated explicitly, which lets the memory-efficient kernel run. The $O(n^2)$ -memory math kernel is disabled. In every forward pass, logits are computed only for the last position, which avoids full-vocabulary logits over long prompts.

Equivalence check. A cached request computes its short uncached suffix with the first path, while full recomputation runs the whole prompt through the second. The two paths are therefore numerically different in bf16, and greedy outputs could in principle diverge near ties. For each model we re-decode the QL and QB queries of the first haystack at every length without reuse (30 pairs; Appendix B) and compare the output strings with the cached ones. Outputs are identical for all 30 pairs of Qwen3-4B and of Phi-4-mini and for 29 of 30 pairs of Qwen2.5-3B. In the one mismatch, a QL query at 16k tokens, the cached path returns a distractor’s number and recomputation returns the correct one, so our cached QL accuracy understates recomputation by one query.

E Reproducibility and Compute

Determinism and logging. No sampling is used. Haystacks are seeded, LongBench items are taken in file order, and the bootstrap and permutation generators use seed 0. Each query writes one JSONL record with its layout, prompt tokens, reused prefix tokens, prefill time, raw output, parsed answer, label or score, and a key that makes interrupted runs resumable without duplicates. One analysis script produces every results table and figure from these logs. We will release the code, prompts, and logs.

Statistical details. For a contrast $b - a$, the paired difference is computed per query (NIAH) or item (LongBench) and summed within clusters. The NIAH primary tests use (length, haystack) clusters with length strata. Each bootstrap replicate resamples clusters with replacement within strata and divides the summed differences by the summed counts. The CI is the 2.5 and 97.5 percentiles over 10,000 replicates. The sign-flip test multiplies the cluster-mean differences by independent random signs, and $p = (1 + \#\{|\text{flipped mean}| \geq |\text{observed mean}|\})/(10,001)$. NIAH analyses keep only (model, length, haystack, needle) combinations with all six cached layouts. We also report an exploratory gap-recovery ratio, $(\text{TP} - \text{QL})/(\text{QB} - \text{QL})$, only when the CI of $\text{QB} - \text{QL}$ excludes zero.

Software and hardware. We use Python 3.12, PyTorch 2.11, Transformers 5.17, NumPy 2.5, and pandas 3.0. All runs use a single 24GB consumer GPU. The Hugging Face Hub checkpoints are Qwen/Qwen2.5-3B-Instruct, Qwen/Qwen3-4B-Instruct-2507, and microsoft/Phi-4-mini-instruct.

Compute. No model is trained. Each block runs under a wall-clock cap, checked before each haystack or item. The NIAH caps are 24 minutes for Qwen3-4B and 20 minutes each for the other two models. The LongBench caps are 22 minutes for Qwen3-4B and 20 minutes each for the other two models, and the exploratory hard NIAH variant has caps of 14 minutes (Qwen3-4B) and 10 minutes (Qwen2.5-3B). The latency block (Qwen3-4B, three lengths, three layouts, eight questions, three repetitions) is uncapped. The caps sum to 150 minutes. Realized sample sizes appear with each table.

F Additional Results

Table 5: NIAH accuracy (%) by model, document length, and layout (40 queries per cell, 8 haystacks).

Model	Length	QL	TP	TP-POST	IRR	QF	QB
Qwen2.5-3B	3.5k	97.5	100.0	100.0	97.5	87.5	100.0
Qwen2.5-3B	8k	97.5	97.5	95.0	95.0	87.5	100.0
Qwen2.5-3B	16k	92.5	92.5	100.0	87.5	57.5	100.0
Phi-4-mini	3.5k	90.0	95.0	82.5	90.0	72.5	95.0
Phi-4-mini	8k	100.0	100.0	100.0	100.0	82.5	100.0
Phi-4-mini	16k	97.5	95.0	95.0	100.0	57.5	97.5
Qwen3-4B-2507	3.5k	100.0	100.0	100.0	100.0	100.0	100.0
Qwen3-4B-2507	8k	100.0	100.0	100.0	100.0	100.0	100.0
Qwen3-4B-2507	16k	100.0	100.0	100.0	100.0	100.0	100.0

Table 6: NIAH error taxonomy (% of queries): correct, returned another needle’s value (distractor), returned an unseen 7-digit number (hallucination), or no 7-digit number (format).

Model	Layout	Correct	Distractor	Halluc.	Format
Qwen2.5-3B	QL	95.8	4.2	0.0	0.0
Qwen2.5-3B	TP	96.7	3.3	0.0	0.0
Qwen2.5-3B	TP-POST	98.3	1.7	0.0	0.0
Qwen2.5-3B	IRR	93.3	6.7	0.0	0.0
Qwen2.5-3B	QF	77.5	22.5	0.0	0.0
Qwen2.5-3B	QB	100.0	0.0	0.0	0.0
Phi-4-mini	QL	95.8	3.3	0.8	0.0
Phi-4-mini	TP	96.7	3.3	0.0	0.0
Phi-4-mini	TP-POST	92.5	6.7	0.8	0.0
Phi-4-mini	IRR	96.7	2.5	0.8	0.0
Phi-4-mini	QF	70.8	29.2	0.0	0.0
Phi-4-mini	QB	97.5	2.5	0.0	0.0
Qwen3-4B-2507	QL	100.0	0.0	0.0	0.0
Qwen3-4B-2507	TP	100.0	0.0	0.0	0.0
Qwen3-4B-2507	TP-POST	100.0	0.0	0.0	0.0
Qwen3-4B-2507	IRR	100.0	0.0	0.0	0.0
Qwen3-4B-2507	QF	100.0	0.0	0.0	0.0
Qwen3-4B-2507	QB	100.0	0.0	0.0	0.0

Table 7: LongBench scores by task and layout (qa-F1 for hotpotqa and multifielddqa_en; retrieval score for passage_retrieval_en). n is the number of items with all layouts completed.

Model	Task	n	QL	TP	IRR	QF	QB
Qwen2.5-3B	passage_retrieval_en	40	57.5	75.0	50.0	57.5	55.0
Qwen2.5-3B	hotpotqa	40	37.8	39.3	32.8	16.8	42.4
Qwen2.5-3B	multifielddqa_en	40	50.3	52.1	47.1	36.2	43.2
Phi-4-mini	passage_retrieval_en	40	95.0	95.0	95.0	80.8	85.0
Phi-4-mini	hotpotqa	40	24.7	26.9	24.5	24.6	28.9
Phi-4-mini	multifielddqa_en	40	52.3	56.2	56.4	35.5	46.8
Qwen3-4B-2507	passage_retrieval_en	40	100.0	100.0	100.0	70.0	80.0
Qwen3-4B-2507	hotpotqa	40	39.3	43.9	40.2	41.5	48.6
Qwen3-4B-2507	multifielddqa_en	40	46.1	53.1	47.5	43.2	43.9

Table 8: Prefill latency (seconds, median of 3 interleaved repetitions) for 8 questions about one document. “First” is the first question; “rest” is the mean over the remaining 7. QL and TP reuse the document KV cache; QB recomputes every prompt. Speedup is QB total over QL total.

Document	QL first	QL rest	TP rest	QB rest	QL total	QB total	Speedup
4k	0.87	0.040	0.041	0.75	1.14	6.05	5.3×
8k	2.06	0.050	0.052	1.89	2.42	15.31	6.3×
16k	5.47	0.075	0.075	5.23	6.07	41.57	6.9×

Table 9: Exploratory NIAH variant with shared-word keys: accuracy (%) by model, document length, and layout (30 queries per cell, 6 haystacks).

Model	Length	QL	TP	TP-POST	IRR	QF	QB
Qwen2.5-3B	8k	86.7	96.7	80.0	86.7	70.0	96.7
Qwen2.5-3B	16k	83.3	93.3	73.3	83.3	46.7	83.3
Qwen3-4B-2507	8k	96.7	100.0	96.7	96.7	100.0	93.3
Qwen3-4B-2507	16k	100.0	100.0	100.0	100.0	100.0	100.0